

Code Review — layer business (auth prin Kong + Swagger)

Repo `nimigeanconstantinon/ms-gitops` · branch `master` · 2026-07-22 · stare FINALĂ (post-fix)

✓

Toate findings-urile rezolvate în repo. Lanțul de auth e acum aliniat la car-platform: intrare prin Kong, gated de oauth2-proxy; Swagger cu login-ul lui Keycloak. Rămâne **un singur pas manual de bootstrap** (sigilarea secretului) — vezi secțiunea finală.

Stare findings

#	Fișier	Problemă	Stare
B1	<code>app-kong.yaml:20</code>	Path <code>valueFiles</code> fără <code>rsk/</code>	rezolvat de Constantin
M1	<code>realm-rsk.yaml</code>	Client <code>spring-api</code> cu secret în clar, neutilizat	scos
M2	<code>oauth2-proxy/*</code> , <code>realm-rsk.yaml</code>	oauth2-proxy referențiat dar inexistent → auth stricat	implementat
M3	<code>data-service/values.yaml:44</code>	Ingress direct ocolea poarta de auth	dezactivat
C1	<code>root.yaml:14</code>	Comentariu leftover	scos
C2	<code>charts/microservice/templates/</code>	Fără <code>_helpers.tpl</code> / label-uri standard / <code>NOTES.txt</code>	deschis (low)
C3	<code>realm-rsk.yaml</code>	<code>registration-service</code> tot cu secret în clar (<code>my-secret-keycloak</code>)	deschis (low)

Lanțul de auth (țintă = realizat)

```
Browser -> https://data-service.icode.mywire.org
nginx Ingress (data-service-gateway)
  --auth-url--> oauth2-proxy POARTA 1 (login + grup /admins)
-> kong-proxy -> ruta declarativa Kong
-> data-service :8081 /swagger-ui
    butonul "Authorize" -> Keycloak (client data-service-swagger, PKCE) POARTA 2
-> Bearer token -> data-service valideaza JWT (realm rsk)
```

Două porți diferite: oauth2-proxy = gate de rețea (nici nu ajungi la Kong fără login + grup); data-service-swagger = auth-ul din Swagger pentru a chema API-ul. Amândouă erau necesare — lipsea doar prima.

Detaliu modificări

M2 — oauth2-proxy portat (fără Crossplane)

car-platform folosește Crossplane ca să exporte `client-secret` -ul în cluster. Constantin **nu are Crossplane**, deci adaptat la tooling-ul lui (sealed-secrets):

- `business/rsk/oauth2-proxy/` — `deployment.yaml` + `service.yaml` + `ingress.yaml` (host `data-service.*`, path `/oauth2`).
- `argo-apps/app-oauth2-proxy.yaml` — Application wave 4.
- Realm: client `oauth2-proxy` confidential **fără secret în fișier** (Keycloak îl generează) + mapper `groups` + grup `/admins` + `admin` pus în grup.

De ce fără secret în YAML: ca să nu repet greșeala M1. Secretul stă într-un `SealedSecret` pe care îl produci tu (nu poate fi sigilat de review — e legat de cheia cluster-ului).

M3 — ușa directă închisă

values.yaml:44	Before	After
ingress	<code>enabled: true</code> (host <code>data.*</code> , fără auth)	<code>enabled: false</code> — singura ușă e Kong

⚠ Pasul manual rămas (bootstrap secret)

oauth2-proxy citește `oauth2-proxy-keycloak` (ns `business`). Pași în `business/rsk/oauth2-proxy/README.md`:

1. **Re-importă realmul** — `KeycloakRealmImport` e create-only: `kubect1 -n auth delete keycloakrealmimport rsk-realm` (Argo îl recrează).
2. Ia `client-secret` -ul generat de Keycloak pentru `oauth2-proxy`.
3. `openssl rand -base64 32` → `cookie-secret`.
4. `kubeseal` → `business/rsk/oauth2-proxy/sealed-secret.yaml` → commit.

Până atunci pod-ul `oauth2-proxy` stă în `CreateContainerConfigError` — normal.

Q&A — de verificat după deploy

1. `curl -sI https://data-service.icode.mywire.org/swagger-ui/` întoarce **302 spre Keycloak** (nu 200 direct)? Dacă dă 200, poarta 1 nu prinde.
2. Login cu `admin/admin` (în `/admins`) trece, iar `test` (fără grup) e respins? Asta validează mapper-ul `groups` + `--allowed-group`.
3. Mai vrei `registration-service` cu secret în clar (C3)? Dacă da, măcar sealed; dacă nu, scoate-l ca la `spring-api`.

Toate fișierele validate YAML. Git neatins — commit/push manual. Review-only override aprobat pentru acest repo.